CREATE: A Closed-Loop Reward Editing Agent with Training Evidence for Reinforcement Learning

Anonymous Authors
Anonymous Submission

I. Introduction

Reinforcement learning (RL) acquires decision policies through interaction with an environment and has been applied to robotics, autonomous systems, games, and complex decision making. The reward function is the interface between task intent and policy optimization: it must express the desired behavior while providing informative and learnable feedback. A semantically plausible reward can nevertheless fail because of sparse signals, conflicting components, inappropriate scales, restrictive gates, or exploitable proxies, thereby degrading learning efficiency and final behavior quality [1]–[4].

Reward engineering is therefore an iterative process of task analysis, reward implementation, policy training, behavior inspection, and revision. A completed training run produces more than task return. Episode length, termination patterns, behavioral outcomes, component activation, component scale, and their evolution during training can reveal how a reward program shapes the learned policy. Converting these heterogeneous records into a precise reward modification, however, still requires substantial human analysis.

Code-capable large language models (LLMs) provide a new route to automating this process. Language to Rewards and Text2Reward translate task descriptions and environment interfaces into executable reward programs [5], [6]; Eureka combines reward-code generation with evolutionary optimization and reward reflection [7]; subsequent work incorporates dynamic feedback, structured reasoning, human feedback, and diagnostic refinement [8]–[11]. These studies establish that LLMs can generate and iteratively improve rewards, while also showing that reward quality becomes observable only after the resulting policy has been trained.

In parallel, language models are increasingly organized as agents that reason over observations, interact with external tools, and retain experience across trials. ReAct interleaves reasoning and environmental actions, whereas Reflexion and Self-Refine use feedback from previous attempts to improve later decisions [12]–[14]. These developments motivate an agent-oriented view of reward engineering in which policy training supplies observations and reward-code revision constitutes an external action.

This paper explores that view through CREATE, a *Closed-Loop Reward Engineering Agent with Training Evidence*. CREATE places a reward-engineering agent above an inner RL policy learner. After each policy-training run, a diagnostic subagent invokes designated evidence analyzers to organize native-task outcomes, termination behavior, reward-component statistics, and training dynamics. A reward editing agent then identifies one principal reward semantic, selects a repair level, and applies a semantically localized reward edit. The next policy-training run evaluates the consequence of that action, while an intervention memory retains the sequence of observations, hypotheses, edits, predictions, and outcomes. A separate best archive preserves the highest-performing executable reward program.

CREATE is evaluated on LunarLander-v3 and BipedalWalker-v3, covering discrete- and continuous-action control. The study analyzes multi-seed reward evolution, representative evidence-to-edit traces, and an unconstrained-revision ablation. The results reported in the completed manuscript examine whether structured observation and semantically localized reward actions can turn initially unsuccessful reward programs into task-solving objectives within a bounded number of policy-training evaluations.

II. Related Work

A. Reward Engineering for Reinforcement Learning

Reward shaping augments a task objective with intermediate signals that improve credit assignment and exploration. Potential-based shaping identifies a class of transformations that preserves the optimal policy [2], while reward machines expose task structure through explicit automata [15]. Inverse reward design treats a specified reward as imperfect evidence of designer intent [3]. Related approaches infer objectives from demonstrations or preferences, or construct intrinsic rewards from novelty and prediction error. These methods reduce direct manual tuning, but typically require designer-specified features, demonstrations, preference data, or a predefined reward representation.

Reward functions have also been treated as objects of search and optimization. Evolutionary methods, meta-gradients, and behavior-based objectives adjust reward parameters or structures according to policy-training outcomes. This transition from manually fixed rewards to revisable reward programs provides the foundation for language-model-based reward engineering.

B. LLM-Based Reward Generation and Refinement

LLMs can map natural-language objectives and environment interfaces to executable reward code. Language to Rewards generates dense reward programs for robotic skills, and Text2Reward produces interpretable free-form rewards

for manipulation and locomotion [5], [6]. Eureka extends generation to evolutionary reward-code optimization and uses component feedback to guide subsequent candidates [7]. REvolve incorporates human feedback into reward evolution, while CARD combines reward generation with dynamic feedback and trajectory-level evaluation [8], [9].

Iterative refinement has also been organized through explicit reasoning. Self-Refine demonstrates that an LLM can improve an output through repeated feedback without parameter updates [14]. The CoT reward-engineering framework decomposes task understanding, reward-component construction, performance analysis, behavioral analysis, and improvement planning into a structured reasoning process [10]. Diagnostic-driven refinement further treats reward failure as a debugging problem and uses training diagnostics to guide targeted revision, while identifying calibration limits in continuous control [11]. Collectively, these methods show that training feedback can support reward revision beyond one-shot generation.

C. LLM Agents and Agentic Reward Engineering

Language-agent research studies how LLMs can maintain state and act in an external process. ReAct couples reasoning with tool-mediated actions, and Reflexion stores verbal feedback from prior trials as episodic experience [12], [13]. Tool use, specialized subagents, and persistent memory have subsequently enabled longer autonomous workflows.

Agentic mechanisms are also entering reward design. RF-Agent formulates reward construction as sequential decision making and uses Monte Carlo tree search to organize candidate generation and historical feedback [16]. STRIDE applies agentic engineering to automate reward design, deep-RL training, and feedback optimization for humanoid locomotion [17]. CREATE follows this agentic direction but centers the interaction interface on instrumented policy training. Its diagnostic subagent converts internal training records into observations, and the reward-engineering agent commits to a semantically localized edit whose consequence is measured by the next complete training run.

III. CREATE

CREATE comprises an outer reward-engineering agent and an underlying reinforcement-learning (RL) agent. The RL agent interacts with the environment and learns a policy under the current generated reward, whereas CREATE treats each completed policy-training and native-evaluation run as one reward-design round. As shown in Fig. 1, CREATE observes training evidence, reflects on the current reward design, and executes a validated reward edit.

The resulting observe–reflect–act loop is written as

$$\begin{aligned} O_t &= \Phi_{\mathcal{T}}(\mathcal{D}_t), \\ (H_t, A_t) &= \Psi(R_t, O_t, M_t, B_t), \\ R_{t+1} &= \mathcal{U}(R_t, A_t), \end{aligned} \tag{1}$$

where the training record $\mathcal{D}_t$ is converted into observation O_t through diagnostic tools. CREATE then combines O_t with the current reward R_t, intervention memory M_t, and best archive B_t to produce reflection H_t, reward action A_t, and the next executable reward program R_{t+1}.

A. Task Understanding and Reward Initialization

Given the task description, environment interface, and reward-code constraints, CREATE identifies the principal behavior semantics and instantiates them as named reward components. The initial reward is

$$R_0(s, a, s') = \sum_{i=1}^{m_0} w_i^{(0)} r_i^{(0)}(s, a, s'), \tag{2}$$

where each component represents a behavior semantic such as task progress, stability, target events, or action cost. Named components are logged independently during training, enabling component-level diagnosis in later rounds. Generated code is checked for interface and execution validity before policy training; prompts and validation rules are provided in the supplementary material.

B. Training-Evidence-Driven Observe–Reflect–Act

At iteration t, the underlying RL agent optimizes the generated reward, while CREATE evaluates the learned policy with the environment-native objective:

$$\begin{aligned} \pi_t &= \arg\max_{\pi} \mathbb{E}_{\tau\sim\pi}\left[\sum_{k=0}^{H-1} \gamma^k R_t(s_k, a_k, s_{k+1})\right], \\ J_t &= \frac{1}{N}\sum_{n=1}^{N}\sum_{k=0}^{H_n-1} r_{\text{env}}(s_k^{(n)}, a_k^{(n)}, s_{k+1}^{(n)}). \end{aligned} \tag{3}$$

Thus, the generated reward shapes policy learning, whereas the native objective measures task completion. Each run produces episode outcomes, termination patterns, component activation and scale, temporal dynamics, and differences from the preceding reward version. A diagnostic subagent queries these records through analysis tools and summarizes the evidence for CREATE.

Component statistics serve as diagnostic cues rather than fixed editing rules. Persistent inactivity may indicate an unreachable gate or sparse formulation; excessive activation and magnitude may reveal a dominant proxy; frequent activation with weak contribution may indicate insufficient scale, saturation, or an ineffective mathematical form. CREATE combines these cues with policy outcomes, reward source code, and prior interventions to identify one principal reward semantic for the next edit:

$$A_t = (q_t, \ell_t, \Delta_t), \qquad \text{supp}(\Delta_t) \subseteq \mathcal{C}(q_t), \tag{4}$$

where q_t denotes the selected semantic, ℓ_t the edit level, and $\mathcal{C}(q_t)$ the components and parameters implementing that semantic. An ordinary action may add or remove a component, adjust its weight or gate, or replace its mathematical form. Related components may be adjusted jointly when they express the same semantic, but independent design intents are not mixed within one ordinary revision. L1 calibrates coefficients, thresholds, or gates; L2 adds, removes, or refactors components while preserving the target semantic; and L3 replans the reward structure after repeated local failure.

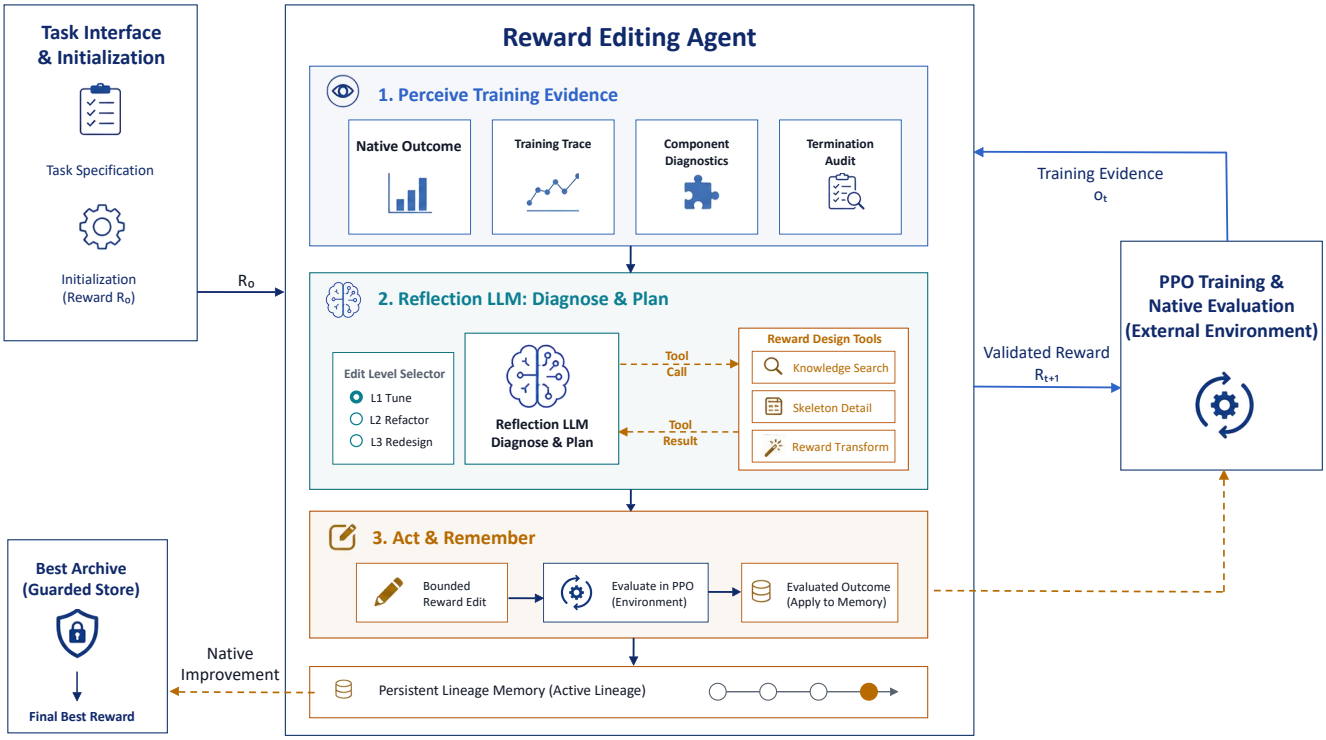

Fig. 1. Overview of CREATE. The outer reward-engineering agent observes training evidence produced by the underlying RL agent, reflects on the current reward design, and performs semantic-localized reward editing. Persistent lineage memory records intervention outcomes, while the best archive retains the reward program with the highest native-task performance.

C. Intervention Memory and Best-Reward Selection

After each round, CREATE records the observation, diagnosed semantic, reward action, expected change, and subsequent outcome in persistent intervention memory. This history supports comparison across rounds and discourages repeated unsuccessful edits. The best archive is maintained separately from the active lineage to prevent later regressions from overwriting a previously discovered solution. After at most K reward evaluations, CREATE returns

$$t^* = \arg\max_{0 \le t < K} J_t, \qquad R^* = R_{t^*}. \tag{5}$$

The selected reward is subsequently retrained and evaluated with independent seeds to assess stability.

References

[1] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.

[2] A. Y. Ng, D. Harada, and S. Russell, "Policy invariance under reward transformations: Theory and application to reward shaping," in *Proceedings of the 16th International Conference on Machine Learning*, 1999, pp. 278–287.

[3] D. Hadfield-Menell, S. Milli, P. Abbeel, S. Russell, and A. Dragan, "Inverse reward design," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[4] D. Amodei, C. Olah, J. Steinhardt, P. Christiano, J. Schulman, and D. Mané, "Concrete problems in AI safety," *arXiv preprint arXiv:1606.06565*, 2016.

[5] W. Yu, N. Gileadi, C. Fu, S. Kirmani, K.-H. Lee, M. G. Arenas, H.-T. L. Chiang, T. Erez, L. Hasenclever, J. Humplik, B. Ichter, T. Xiao, P. Xu, A. Zeng, T. Zhang, N. Heess, D. Sadigh, J. Tan, Y. Tassa, and F. Xia, "Language to rewards for robotic skill synthesis," in *Proceedings of the 7th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 229, 2023, pp. 374–404.

[6] T. Xie, S. Zhao, C. H. Wu, Y. Liu, Q. Luo, V. Zhong, Y. Yang, and T. Yu, "Text2reward: Reward shaping with language models for reinforcement learning," in *International Conference on Learning Representations*, 2024.

[7] Y. J. Ma, W. Liang, G. Wang, D.-A. Huang, O. Bastani, D. Jayaraman, Y. Zhu, L. Fan, and A. Anandkumar, "Eureka: Human-level reward design via coding large language models," in *International Conference on Learning Representations*, 2024.

[8] S. Sun, R. Liu, J. Lyu, J.-W. Yang, L. Zhang, and X. Li, "A large language model-driven reward design framework via dynamic feedback for reinforcement learning," *Knowledge-Based Systems*, vol. 326, p. 114065, 2025.

[9] R. Hazra, A. Sygkounas, A. Persson, A. Loutfi, and P. Z. D. Martires, "REvolve: Reward evolution with large language models using human feedback," in *International Conference on Learning Representations*, 2025.

[10] X. Zhu, J. Du, Q. Fu, and L. Chen, "LLM-based reward engineering for reinforcement learning: A chain of thought approach," in *2025 10th International Conference on Cloud Computing and Big Data Analytics (ICCCBDA)*, 2025.

[11] Y. Wang, Y. Tang, B. Liu, X. Liu, and D. Shang, "When LLM reward design fails: Diagnostic-driven refinement for sparse structured RL," *arXiv preprint arXiv:2605.28918*, 2026.

[12] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations*, 2023.

[13] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[14] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang, S. Gupta, B. P. Majumder,

K. Hermann, S. Welleck, A. Yazdanbakhsh, and P. Clark, "Self-refine: Iterative refinement with self-feedback," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[15] R. T. Icarte, T. Q. Klassen, R. Valenzano, and S. A. McIlraith, "Reward machines: Exploiting reward function structure in reinforcement learning," *Journal of Artificial Intelligence Research*, vol. 73, pp. 173–208, 2022.

[16] N. Gao, X. Zhang, X. Jiang, M. You, M. Zhang, and Y. Deng, "RF-Agent: Automated reward function design via language agent tree search," in *Advances in Neural Information Processing Systems*, vol. 38, 2025.

[17] Z. Wu, J. Lu, Y. Chen, Y. Liu, Y. Zhuang, and L. Hu, "STRIDE: Automating reward design, deep reinforcement learning training and feedback optimization in humanoid robotics locomotion," *arXiv preprint arXiv:2502.04692*, 2025.